

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО» (УНИВЕРСИТЕТ ИТМО)
Факультет «Систем управления и робототехники»

Алгоритмы ориентации в пространстве

Отчёт по лабораторной работе № 2

Работу
выполнил:
Сухов Н. М.
Группа: R3335
Преподаватель:
Каплин А. В.

Санкт-Петербург
2026

Содержание

1. Цель работы	3
2. Постановка задачи	3
3. Модель движения робота	3
4. Модель измерений маячков	4
5. Расширенный фильтр Калмана	4
6. Якобиан модели измерений	5
7. Параметры моделирования	5
8. Оценка качества локализации	6
9. Эксперименты с различными параметрами шумов	7
10. Результаты базового эксперимента	7
11. Результаты эксперимента с увеличенным шумом управления	10
12. Результаты эксперимента с увеличенным шумом измерений	12
13. Сравнение результатов	15
14. Обсуждение результатов	15
15. Вывод	16
16. Исходный код	16

1. Цель работы

Целью лабораторной работы является реализация расширенного фильтра Калмана для локализации дифференциального колесного робота в плоскости. Вектор состояния робота задается в виде

$$\mathbf{x} = [x \quad y \quad \theta]^T,$$

где x и y - координаты центра робота в мировой системе координат, θ - угол ориентации робота.

В ходе работы выполняется моделирование движения робота в среде CoppeliaSim, формирование зашумленных управляющих воздействий и измерений маячков, а также сравнение результатов локализации по одометрии и по расширенному фильтру Калмана.

2. Постановка задачи

В лабораторной работе рассматривается дифференциальный колесный робот Pioneer3DX, движущийся по плоскости. В сцене расположены четыре маячка с известными координатами. Для каждого маячка моделируются измерения дальности и пеленга:

$$\mathbf{z} = [\rho \quad \varphi]^T,$$

где ρ - расстояние от робота до маячка, φ - угол направления на маячок в системе координат робота.

Необходимо реализовать алгоритм EKF, сравнить его с одометрией и истинной траекторией робота, а также оценить качество локализации с помощью RMSE, NIS и NEES.

3. Модель движения робота

В качестве управляющего воздействия используется вектор

$$\mathbf{u} = [v \quad \omega]^T,$$

где v - линейная скорость робота, ω - угловая скорость робота.

Для дифференциального робота линейная и угловая скорости определяются через угловые скорости колес:

$$v = \frac{r}{2} (\omega_R + \omega_L),$$
$$\omega = \frac{r}{L} (\omega_R - \omega_L),$$

где r - радиус колеса, L - расстояние между колесами, ω_L и ω_R - угловые скорости левого и правого колес.

При $\omega \neq 0$ дискретная модель движения за интервал времени Δt имеет вид:

$$x_{k+1} = x_k + \frac{v}{\omega} [\sin(\theta_k + \omega \Delta t) - \sin \theta_k],$$
$$y_{k+1} = y_k - \frac{v}{\omega} [\cos(\theta_k + \omega \Delta t) - \cos \theta_k],$$
$$\theta_{k+1} = \theta_k + \omega \Delta t$$

При $\omega \approx 0$ используется приближение прямолинейного движения:

$$x_{k+1} = x_k + v \Delta t \cos \theta_k,$$
$$y_{k+1} = y_k + v \Delta t \sin \theta_k,$$
$$\theta_{k+1} = \theta_k + \omega \Delta t$$

4. Модель измерений маячков

Пусть маячок имеет известные координаты

$$\mathbf{m}_i = [x_i \ y_i]^T$$

Тогда для состояния робота $\mathbf{x} = [x, y, \theta]^T$ модель измерения дальности и пеленга записывается в виде

$$\begin{aligned}\rho_i &= \sqrt{(x_i - x)^2 + (y_i - y)^2}, \\ \varphi_i &= \text{atan2}(y_i - y, x_i - x) - \theta\end{aligned}$$

Таким образом, нелинейная функция измерения имеет вид:

$$\mathbf{z}_i = h_i(\mathbf{x}) = \begin{bmatrix} \sqrt{(x_i - x)^2 + (y_i - y)^2} \\ \text{atan2}(y_i - y, x_i - x) - \theta \end{bmatrix}$$

5. Расширенный фильтр Калмана

Расширенный фильтр Калмана состоит из двух основных этапов: прогноза и коррекции. На этапе прогноза состояние рассчитывается по модели движения:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_k)$$

Ковариационная матрица ошибки прогноза определяется выражением

$$\mathbf{P}_{k|k-1} = \mathbf{F}_x \mathbf{P}_{k-1|k-1} \mathbf{F}_x^T + \mathbf{F}_u \mathbf{Q} \mathbf{F}_u^T,$$

где $\mathbf{F}_x$ - якобиан модели движения по состоянию, $\mathbf{F}_u$ - якобиан модели движения по управлению, $\mathbf{Q}$ - ковариационная матрица шума управления.

На этапе коррекции рассчитывается невязка измерений:

$$\mathbf{y}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})$$

Ковариация невязки:

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R},$$

где $\mathbf{H}_k$ - якобиан модели измерений, $\mathbf{R}$ - ковариационная матрица шума измерений.

Коэффициент Калмана:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1}$$

Коррекция состояния:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k$$

Коррекция ковариационной матрицы:

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T + \mathbf{K}_k \mathbf{R} \mathbf{K}_k^T$$

6. Якобиан модели измерений

Для маячка с координатами (x_i, y_i) введем обозначения:

$$d_x = x_i - x, \quad d_y = y_i - y, \quad q = d_x^2 + d_y^2$$

Тогда якобиан измерительной модели имеет вид:

$$\mathbf{H} = \begin{bmatrix} -\frac{d_x}{\sqrt{q}} & -\frac{d_y}{\sqrt{q}} & 0 \\ \frac{d_y}{q} & -\frac{d_x}{q} & -1 \end{bmatrix}$$

Данный якобиан используется на этапе коррекции EKF для каждого маячка.

7. Параметры моделирования

Моделирование выполнялось в среде CoppeliaSim. В качестве мобильного робота использовалась модель Pioneer3DX. В сцене были заданы четыре маячка:

$$b_1, b_2, b_3, b_4$$

Скрипт был добавлен как Python Child Script к объекту робота Pioneer3DX. Для записи результатов моделирования формировался CSV-файл, содержащий истинную траекторию робота, одометрическую оценку, оценку EKF, элементы ковариационной матрицы, а также значения критериев NIS и NEES.

В работе использовались следующие параметры робота:

$$r = 0.0975 \text{ м},$$

$$L = 0.331 \text{ м}$$

Время моделирования:

$$T = 120 \text{ с}$$

Начальная ковариационная матрица:

$$\mathbf{P}_0 = \text{diag}(0.05^2, 0.05^2, (5^\circ)^2)$$

Базовая ковариационная матрица шума управления:

$$\mathbf{Q} = \text{diag}(0.018^2, 0.08^2)$$

Базовая ковариационная матрица шума измерений:

$$\mathbf{R} = \text{diag}(0.03^2, (2^\circ)^2)$$

Сцена моделирования на рис. 7.1.

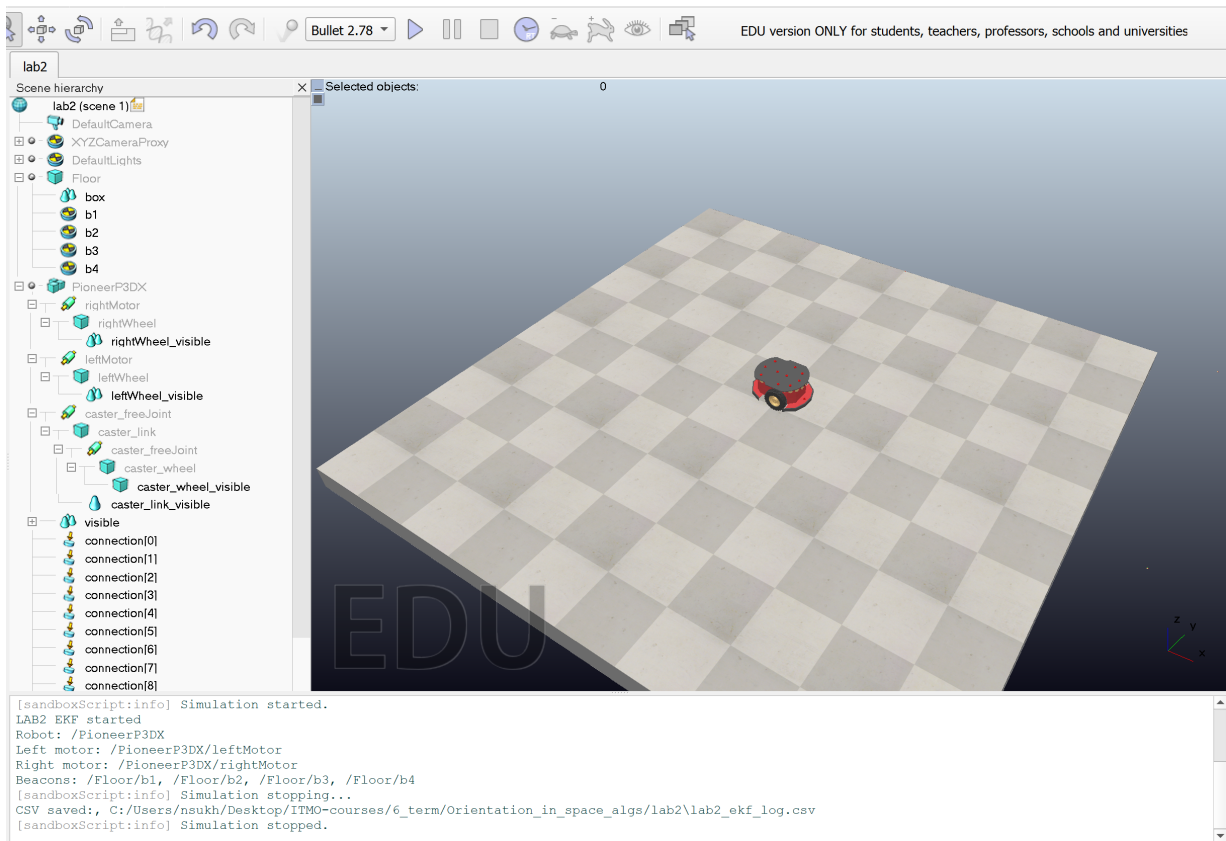


Рисунок 7.1. Сцена

8. Оценка качества локализации

Для оценки точности локализации использовалась среднеквадратическая ошибка положения:

$$RMSE_{pos} = \sqrt{\frac{1}{N} \sum_{k=1}^N [(x_k - \hat{x}_k)^2 + (y_k - \hat{y}_k)^2]}$$

Также использовались критерии NIS и NEES.

Критерий NIS определяется выражением

$$NIS_k = \mathbf{y}_k^T \mathbf{S}_k^{-1} \mathbf{y}_k$$

Он показывает, насколько сильно фактическое измерение отличается от ожидаемого измерения с учетом ковариации невязки.

Критерий NEES определяется выражением

$$NEES_k = \mathbf{e}_k^T \mathbf{P}_k^{-1} \mathbf{e}_k,$$

где

$$\mathbf{e}_k = \hat{\mathbf{x}}_k - \mathbf{x}_{true,k}.$$

Критерий NEES показывает, насколько фактическая ошибка оценки соответствует ковариационной матрице фильтра.

9. Эксперименты с различными параметрами шумов

Были проведены три эксперимента:

1. базовый вариант;
2. вариант с увеличенной ковариацией шума управления Q ;
3. вариант с увеличенной ковариацией шума измерений R .

В варианте с увеличенным шумом управления использовалась матрица:

$$\mathbf{Q}_{high} = \text{diag}(0.05^2, 0.20^2)$$

В варианте с увеличенным шумом измерений были заданы:

$$\sigma_\rho = 0.10 \text{ м},$$

$$\sigma_\varphi = 6^\circ$$

10. Результаты базового эксперимента

На рис. 10.1 представлены истинная траектория робота, траектория по одометрии и траектория, полученная с помощью ЕКФ. Видно, что одометрия постепенно накапливает ошибку и заметно отклоняется от истинной траектории. В то же время оценка ЕКФ практически совпадает с истинной траекторией.

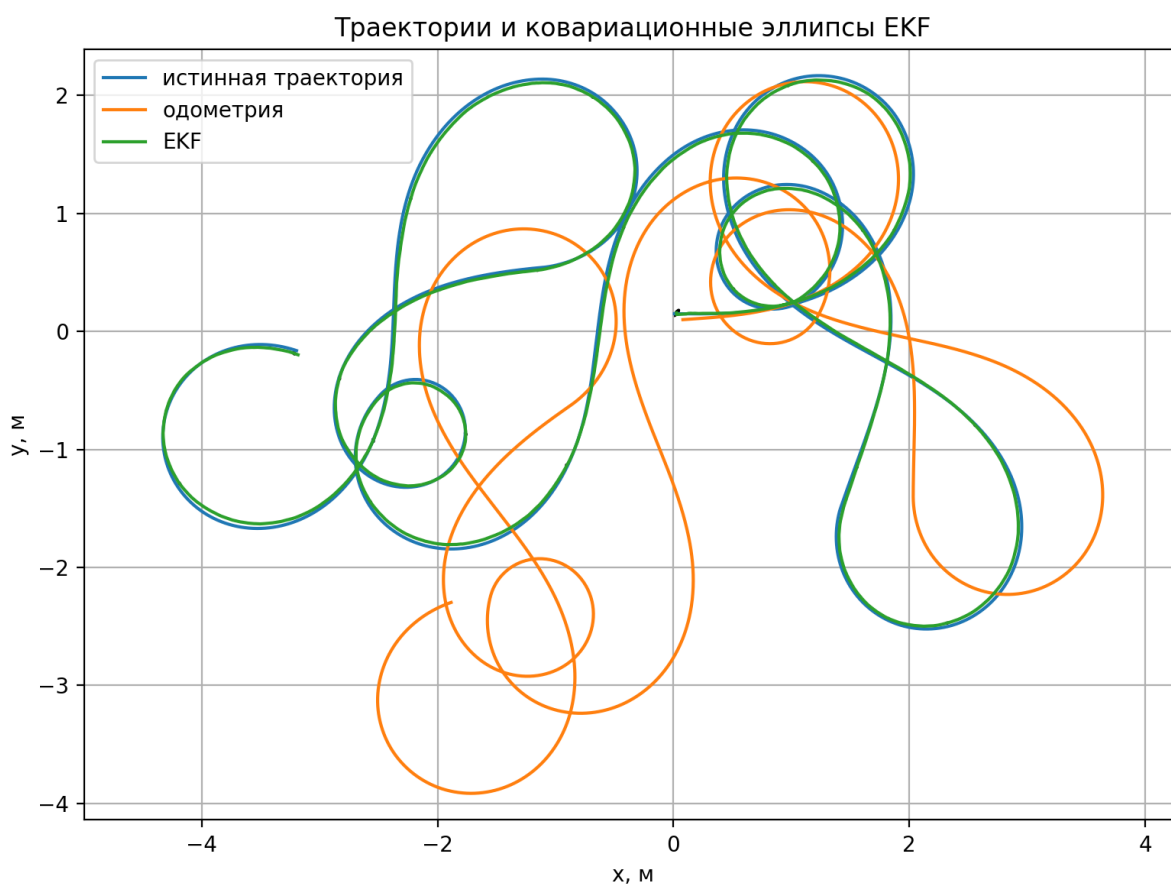


Рисунок 10.1. Траектории робота и ковариационные эллипсы ЕКФ для базового эксперимента

На рис. 10.2 показана ошибка положения для одометрии и EKF. Ошибка одометрии достигает нескольких метров, тогда как ошибка EKF остается малой на всем интервале моделирования.

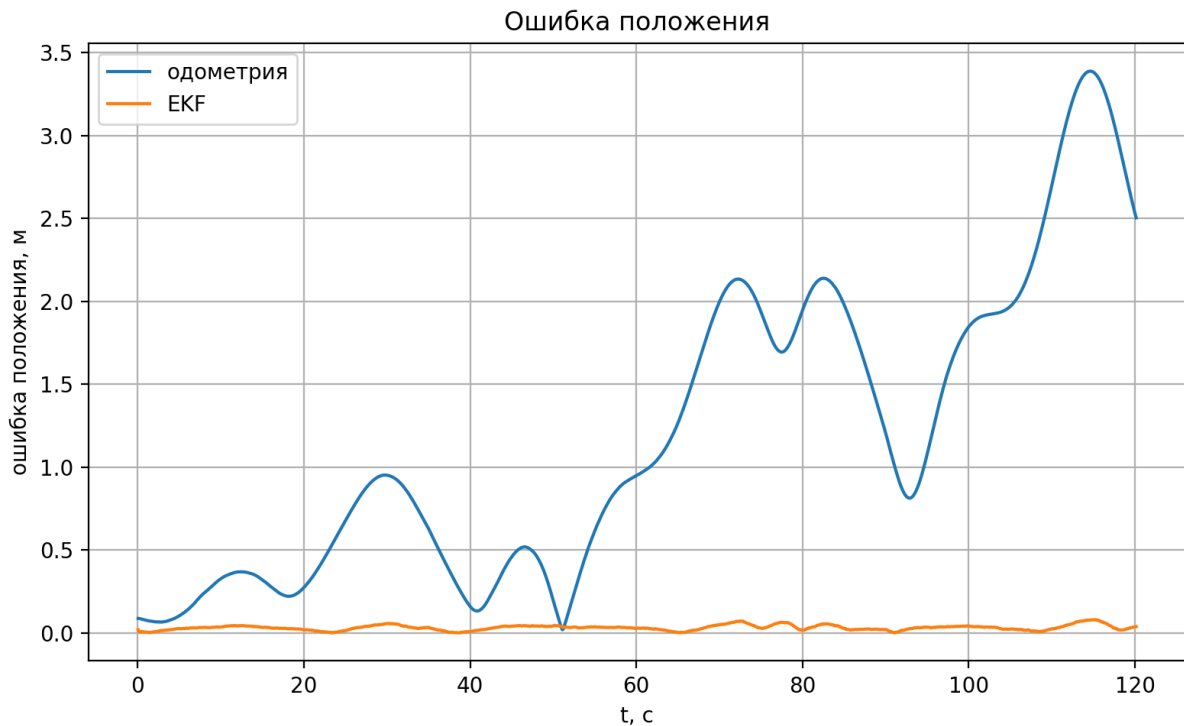


Рисунок 10.2. Ошибка положения для базового эксперимента

На рис. 10.3 представлено изменение стандартных отклонений σ_x , σ_y , σ_θ . В начале моделирования наблюдается резкое уменьшение неопределенности, что связано с коррекцией оценки по измерениям маячков. Далее значения стабилизируются.

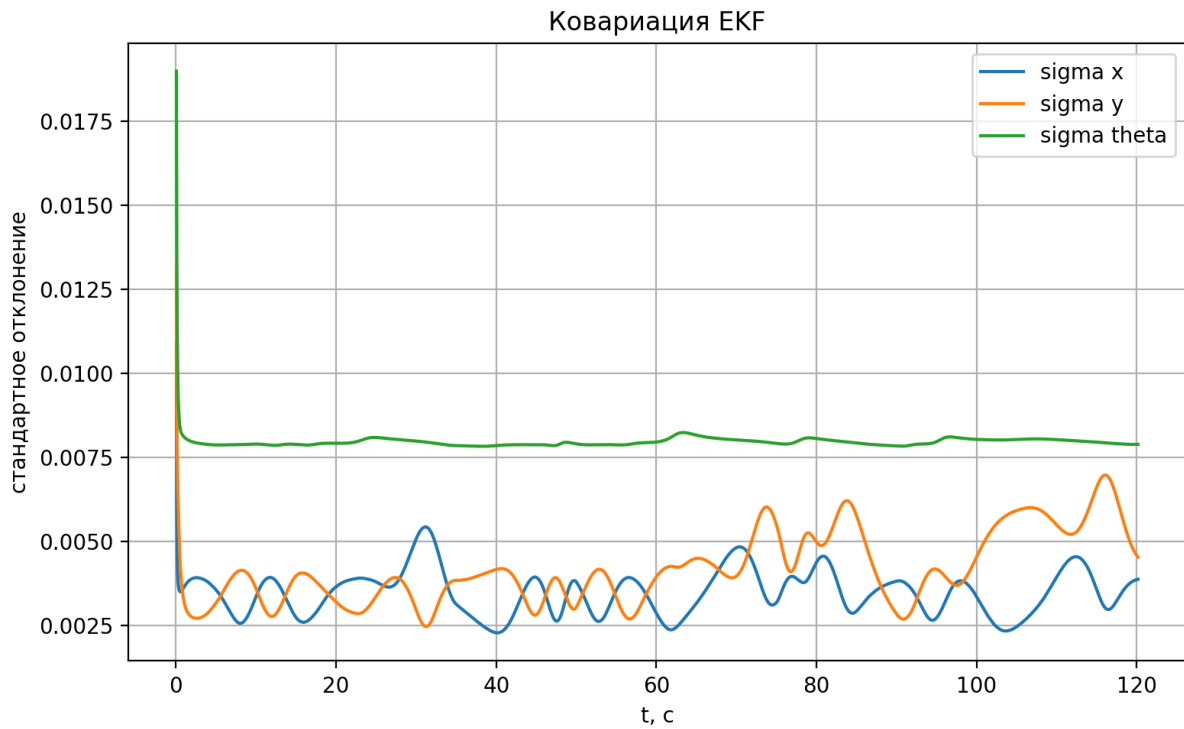


Рисунок 10.3. Ковариация EKF для базового эксперимента

На рис. 10.4 представлены значения NIS и NEES. Значение NIS находится в диапазоне нескольких единиц, что соответствует ожидаемому порядку величины для измерений дальности и пеленга. Значение NEES существенно выше размерности состояния, что указывает на избыточную уверенность фильтра в собственной оценке.

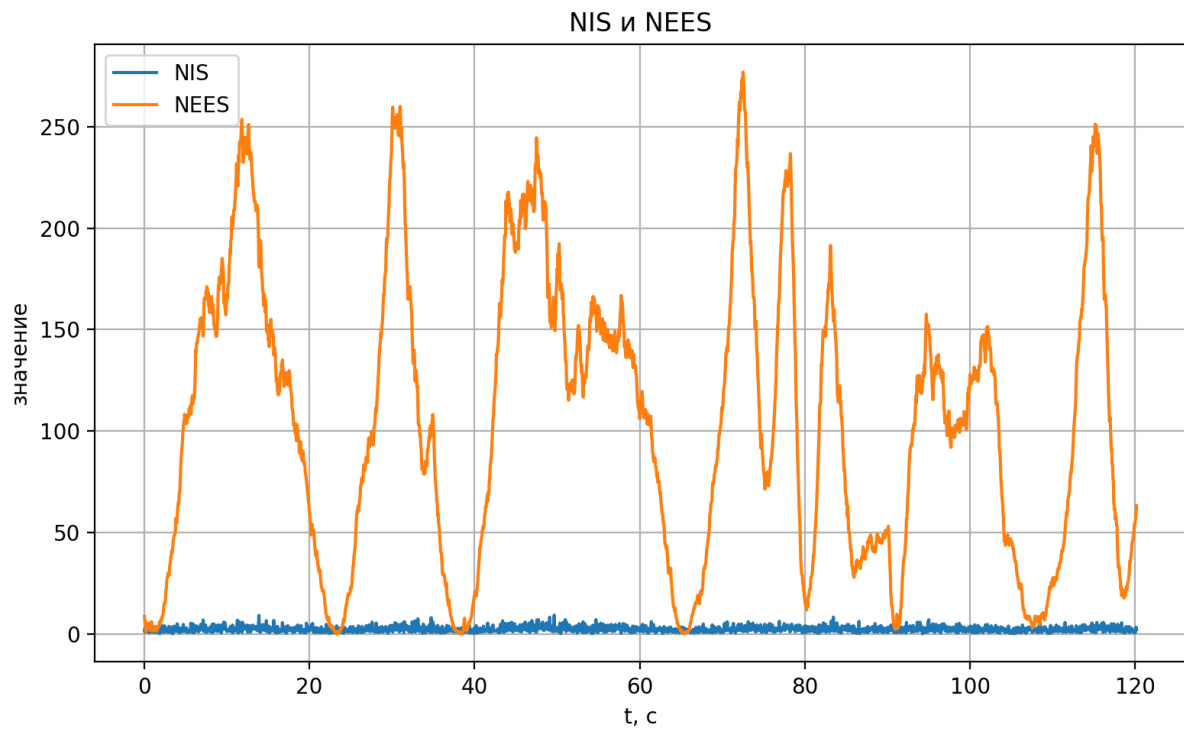


Рисунок 10.4. NIS и NEES для базового эксперимента

11. Результаты эксперимента с увеличенным шумом управления

На рис. 11.1-11.4 представлены результаты эксперимента с увеличенной матрицей Q . Увеличение Q означает, что фильтр меньше доверяет модели движения и учитывает большую неопределенность управляющих воздействий.

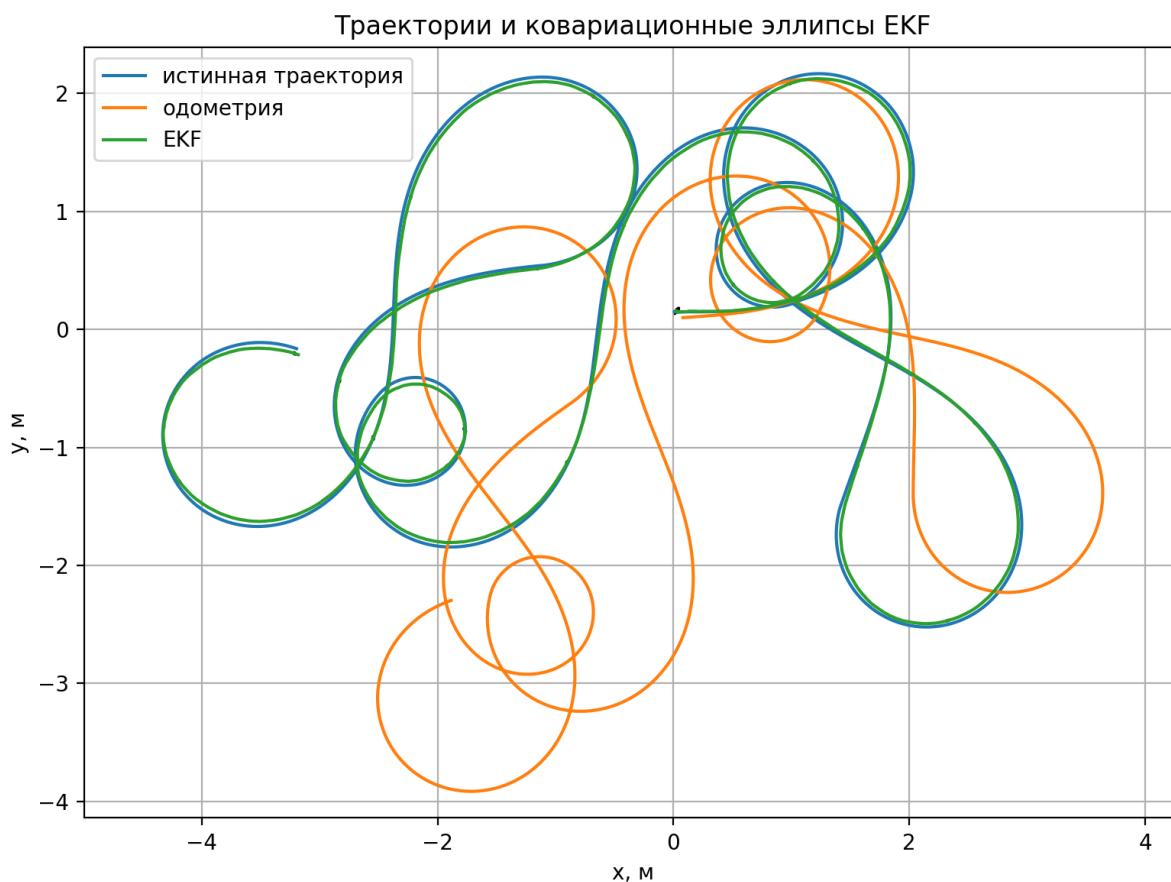


Рисунок 11.1. Траектории робота и ковариационные эллипсы EKF при увеличенном Q

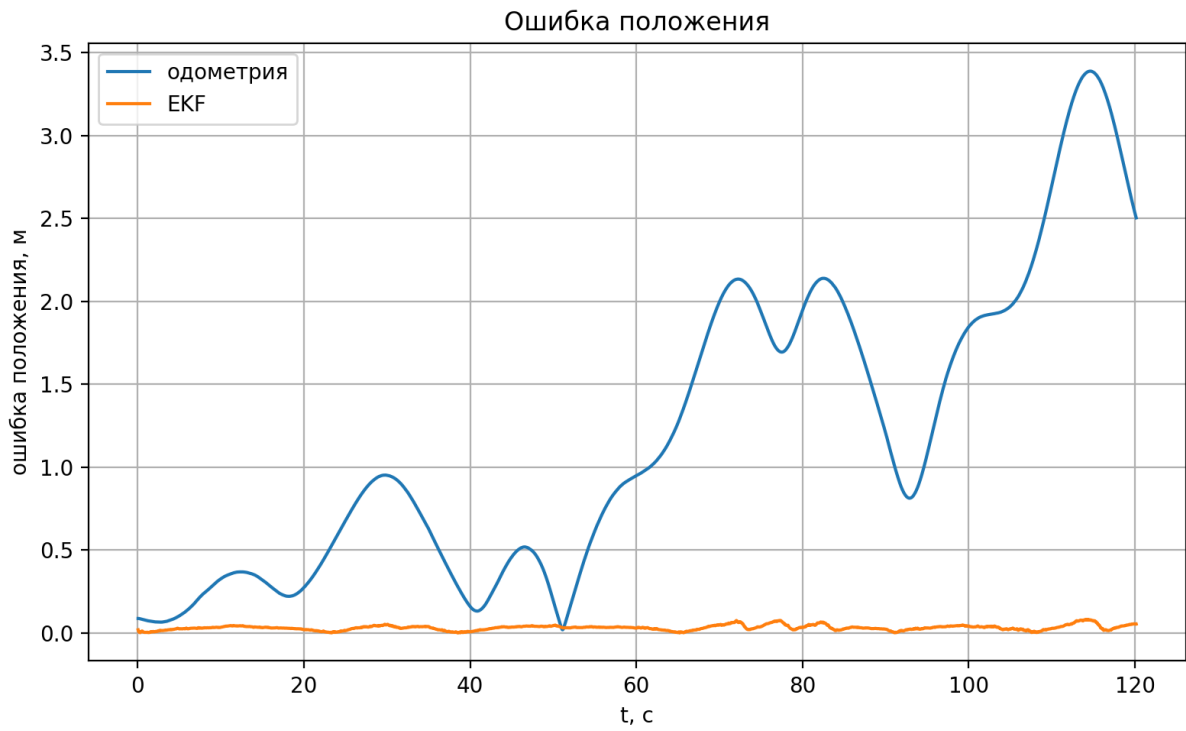


Рисунок 11.2. Ошибка положения при увеличенном Q

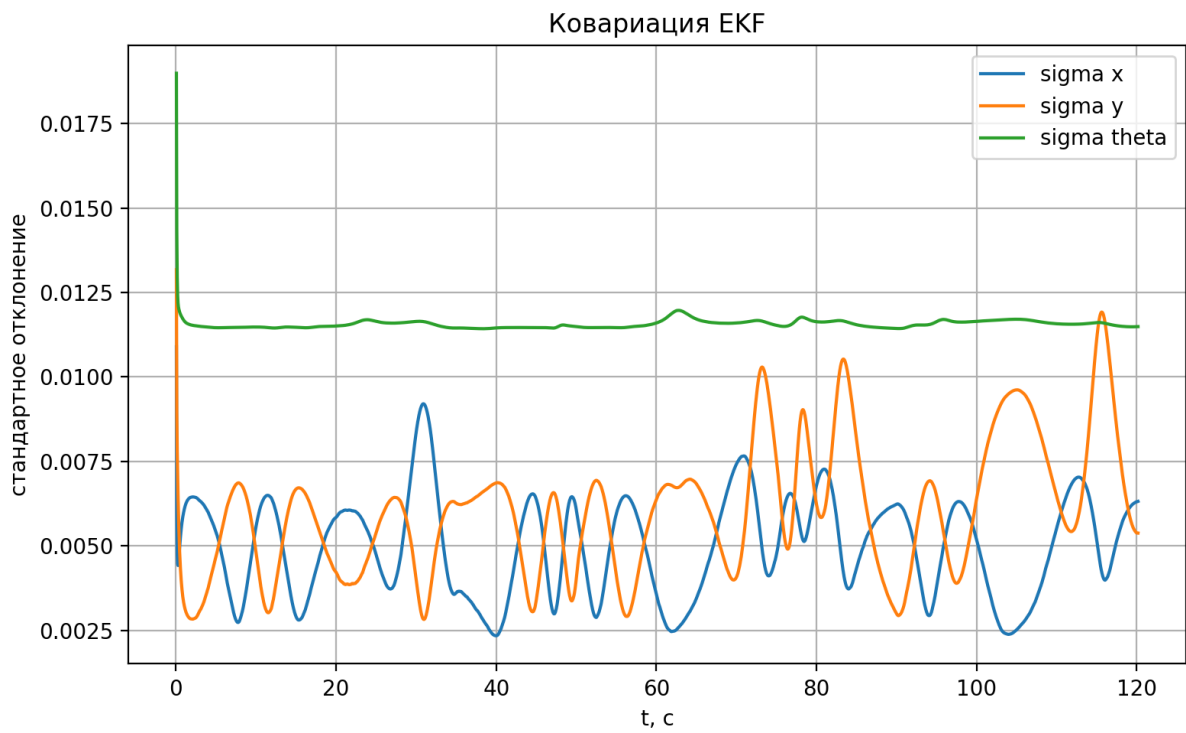


Рисунок 11.3. Ковариация EKF при увеличенном Q

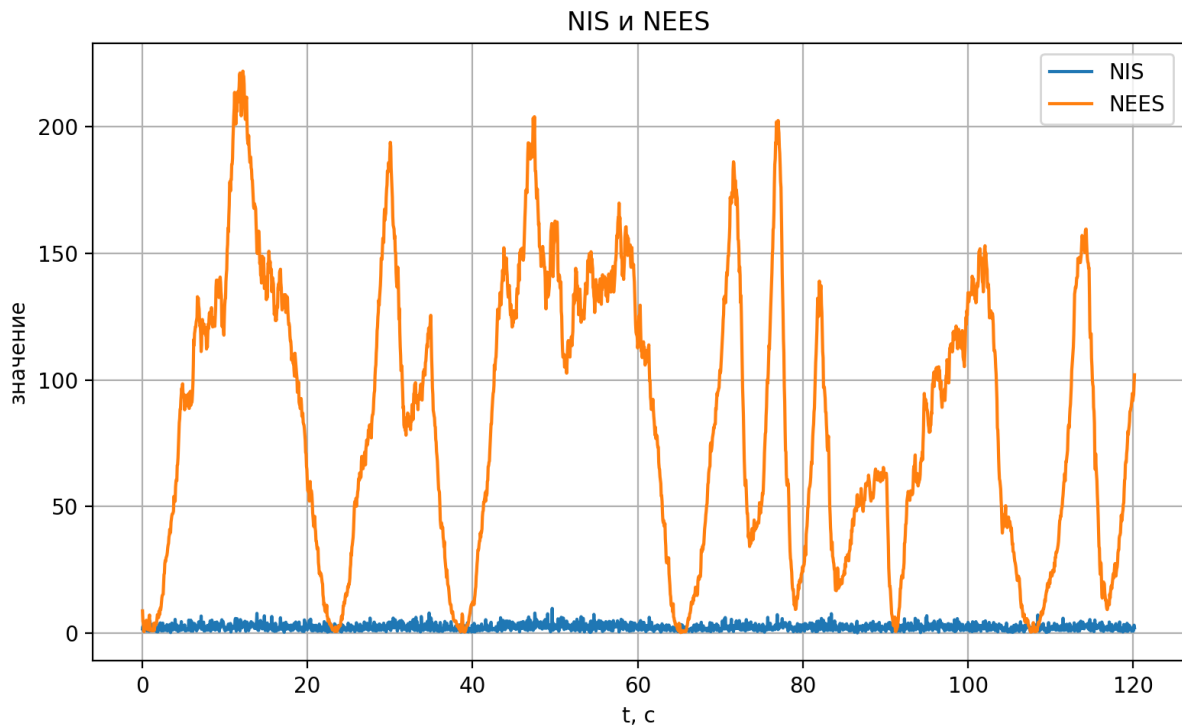


Рисунок 11.4. NIS и NEES при увеличенном Q

Из графиков видно, что увеличение Q практически не изменило вид траектории ЕКФ. Это объясняется тем, что в каждом шаге фильтр получает измерения от четырех маячков, поэтому оценка быстро корректируется и остается близкой к истинной траектории.

12. Результаты эксперимента с увеличенным шумом измерений

На рис. 12.1-12.4 представлены результаты эксперимента с увеличенной матрицей R . Увеличение R означает, что фильтр меньше доверяет измерениям маячков.

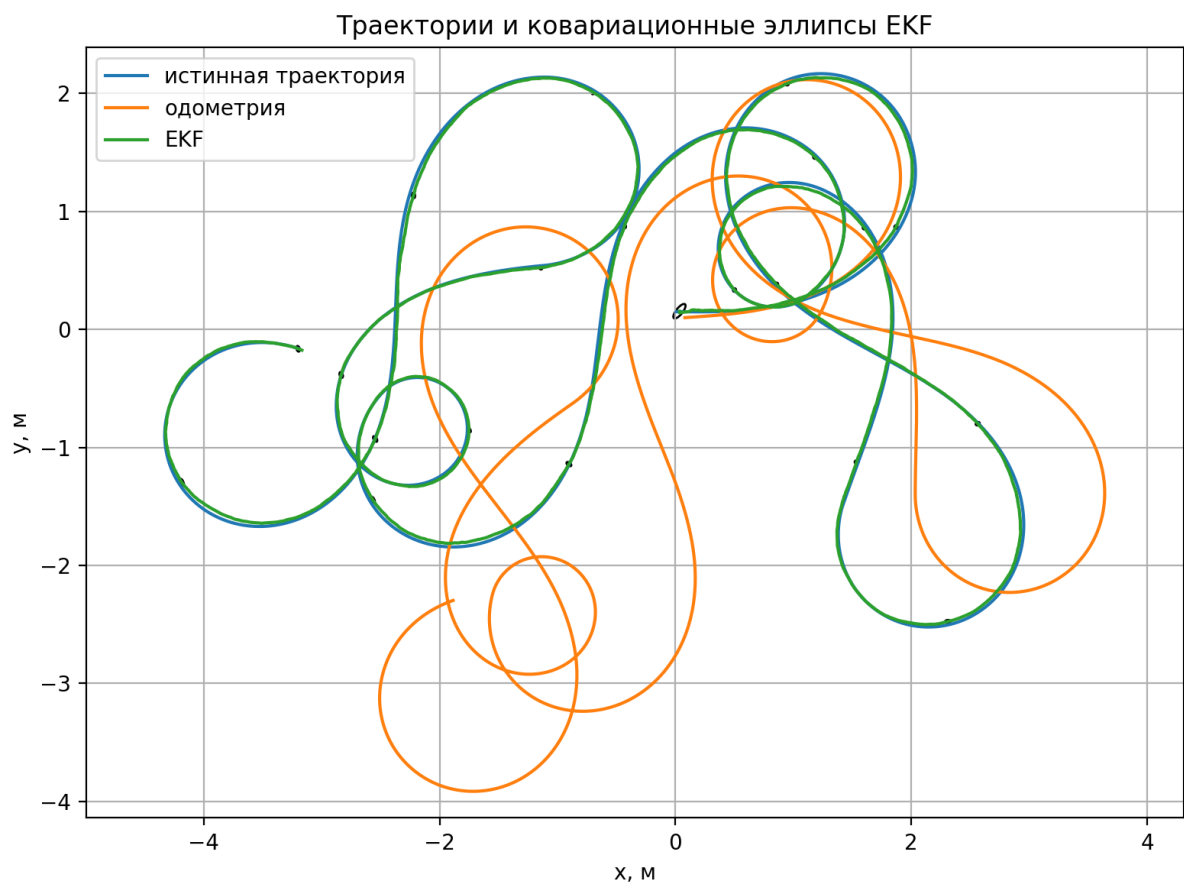


Рисунок 12.1. Траектории робота и ковариационные эллипсы EKF при увеличенном R

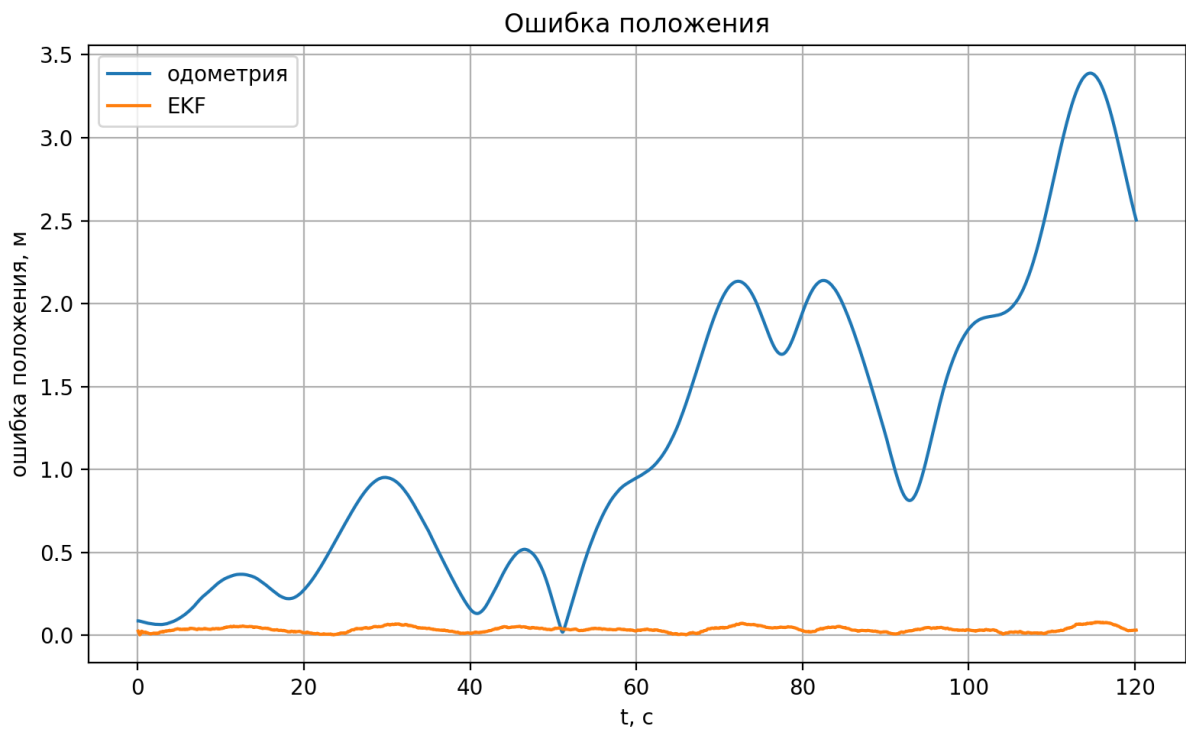


Рисунок 12.2. Ошибка положения при увеличенном R

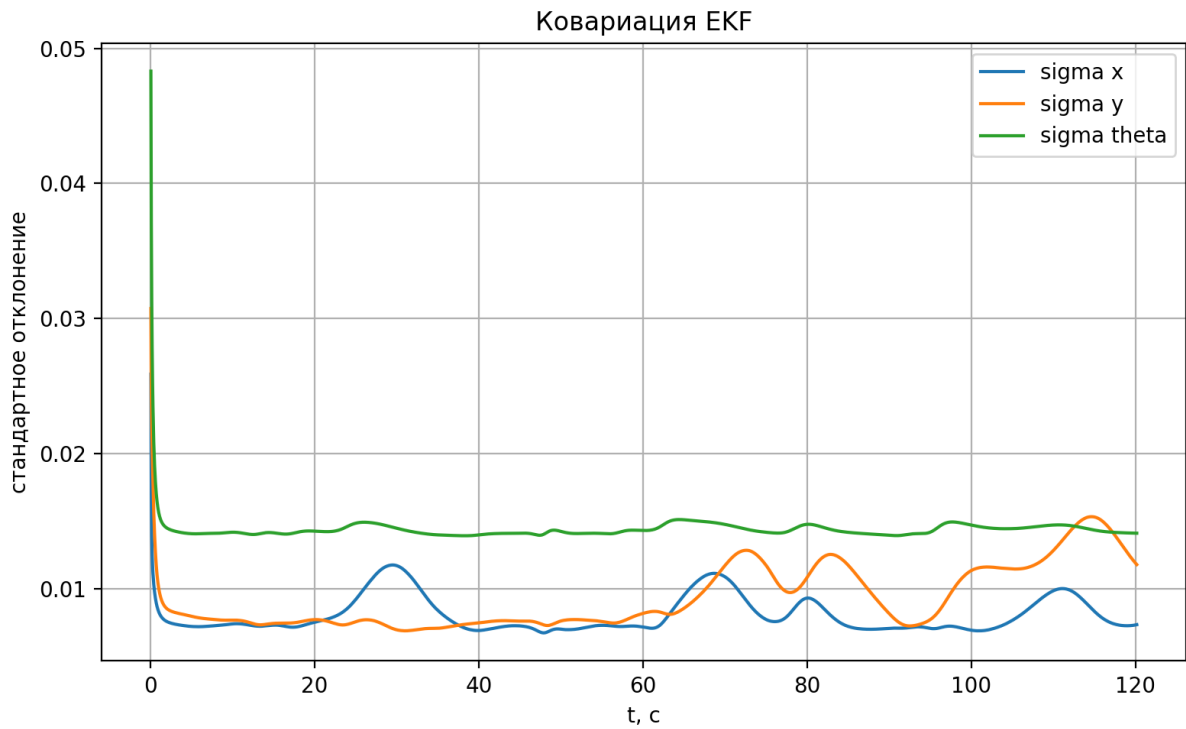


Рисунок 12.3. Ковариация EKF при увеличенном R

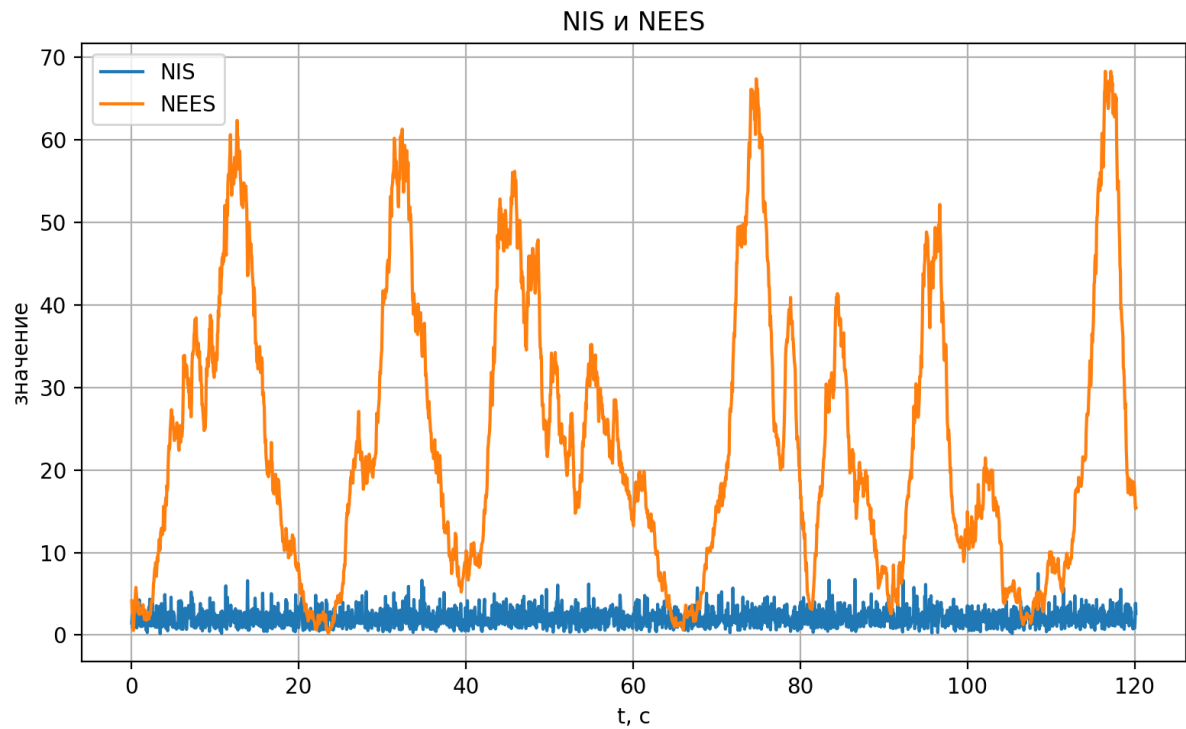


Рисунок 12.4. NIS и NEES при увеличенном R

По сравнению с базовым вариантом ковариации в эксперименте с увеличенным R стали больше. Это соответствует физическому смыслу: при большем шуме измерений фильтр менее уверен в корректирующей информации от маячков.

13. Сравнение результатов

Сводные результаты экспериментов приведены в табл. 13.1.

Таблица 13.1

Сравнение качества локализации

Эксперимент	$RMSE_{EKF}$, м	$RMSE_{odom}$, м	$RMSE_{\theta,EKF}$, рад	$RMSE_{\theta,odom}$, рад	Средний NIS	Средний $NEES$
Базовый	0.0353	1.4866	0.00875	0.4769	2.5045	104.4095
Увеличенный Q	0.0352	1.4866	0.01142	0.4769	2.3655	84.5331
Увеличенный R	0.0397	1.4866	0.01286	0.4769	2.0636	23.4661

Из табл. 13.1 видно, что во всех экспериментах EKF существенно превосходит одометрию по точности оценки положения. Средняя ошибка EKF составляет примерно 3.5-4.0 см, тогда как ошибка одометрии составляет около 1.49 м.

Значения $RMSE_{odom}$ совпадают во всех экспериментах, так как траектория движения робота и зашумленные одометрические данные оставались одинаковыми. Изменялись только параметры фильтра Q и R .

Визуально траектории EKF для трех экспериментов почти совпадают. Это связано с тем, что робот двигался по одной и той же траектории, а фильтр на каждом шаге получал измерения от четырех маячков. Поэтому даже при изменении Q и R оценка EKF оставалась близкой к истинной траектории.

14. Обсуждение результатов

Базовый вариант обеспечивает минимальную ошибку положения:

$$RMSE_{EKF} = 0.0353 \text{ м}$$

Однако значение NEES для базового варианта равно

$$NEES = 104.4095$$

Это существенно больше размерности состояния, равной 3. Следовательно, фильтр является избыточно уверенным в собственной оценке, то есть ковариационная матрица P оказывается слишком малой по сравнению с фактической ошибкой.

При увеличении Q ошибка положения практически не изменилась:

$$RMSE_{EKF} = 0.0352 \text{ м}$$

При этом значение NEES снизилось до

$$NEES = 84.5331$$

Это означает, что увеличение шума управления немного уменьшило самоуверенность фильтра, однако полностью проблему консистентности не устранило.

При увеличении R ошибка положения немного увеличилась:

$$RMSE_{EKF} = 0.0397 \text{ м}$$

При этом значение NEES существенно снизилось:

$$NEES = 23.4661$$

Следовательно, увеличение шума измерений сделало фильтр менее самоуверенным и более консистентным, хотя значение NEES все еще остается выше идеального уровня.

Таким образом, увеличение R в данном эксперименте сильнее повлияло на консистентность фильтра, чем увеличение Q .

15. Вывод

В ходе лабораторной работы был реализован расширенный фильтр Калмана для локализации дифференциального колесного робота Pioneer3-DX в среде CoppeliaSim. Для коррекции оценки использовались измерения дальности и пеленга до четырех маячков.

По результатам моделирования установлено, что EKF значительно повышает точность локализации по сравнению с чистой одометрией. Средняя ошибка положения по EKF составила около 0.035-0.040 м, тогда как ошибка одометрии составила около 1.49 м. Следовательно, использование измерений маячков позволяет компенсировать накопление ошибки одометрии.

Проведенные эксперименты с различными значениями матриц Q и R показали, что изменение параметров шумов влияет не только на точность, но и на консистентность фильтра. Увеличение Q почти не изменило точность оценки, но немного уменьшило значение NEES. Увеличение R привело к небольшому росту ошибки EKF, однако существенно уменьшило NEES. Это означает, что при большем шуме измерений фильтр становится менее самоуверенным.

В целом результаты подтверждают работоспособность EKF для задачи локализации колесного робота. Наилучшая точность была получена в базовом варианте и варианте с увеличенным Q , однако с точки зрения консистентности более предпочтительным оказался вариант с увеличенным R .

16. Исходный код

Listing 1: Child script

```
1 import math
2 import os
3 import csv
4 import random
5 import numpy as np
6
7 sim = None
8
9 WHEEL_RADIUS = 0.0975
10 WHEEL_BASE = 0.331
11
12 SIGMA_WHEEL = 0.04
13 SIGMA_RANGE = 0.10
14 SIGMA_BEARING = math.radians(6.0)
15
16 Q_CONTROL = np.diag([0.018 ** 2, 0.08 ** 2])
17 R_BEACON = np.diag([SIGMA_RANGE ** 2, SIGMA_BEARING ** 2])
18 P0 = np.diag([0.05 ** 2, 0.05 ** 2, math.radians(5.0) ** 2])
19
```



```

20 MAX_TIME = 120.0
21
22
23 def wrap_angle(a):
24     return (a + math.pi) % (2.0 * math.pi) - math.pi
25
26
27 def motion_model(x, u, dt):
28     px = x[0]
29     py = x[1]
30     th = x[2]
31
32     v = u[0]
33     w = u[1]
34
35     if abs(w) > 1e-6:
36         px_new = px + (v / w) * (math.sin(th + w * dt) - math.sin(th))
37         py_new = py - (v / w) * (math.cos(th + w * dt) - math.cos(th))
38     else:
39         px_new = px + v * dt * math.cos(th)
40         py_new = py + v * dt * math.sin(th)
41
42     th_new = wrap_angle(th + w * dt)
43
44     return np.array([px_new, py_new, th_new], dtype=float)
45
46
47 def jacobians_motion(x, u, dt):
48     th = x[2]
49     v = u[0]
50     w = u[1]
51
52     Fx = np.eye(3)
53     Fu = np.zeros((3, 2))
54
55     if abs(w) > 1e-6:
56         th2 = th + w * dt
57
58         Fx[0, 2] = (v / w) * (math.cos(th2) - math.cos(th))
59         Fx[1, 2] = (v / w) * (math.sin(th2) - math.sin(th))
60
61         Fu[0, 0] = (math.sin(th2) - math.sin(th)) / w
62         Fu[0, 1] = v * (w * dt * math.cos(th2) - math.sin(th2) + math.sin(th)) /
63             (w ** 2)
64
65         Fu[1, 0] = -(math.cos(th2) - math.cos(th)) / w
66         Fu[1, 1] = v * (w * dt * math.sin(th2) + math.cos(th2) - math.cos(th)) /
67             (w ** 2)
68
69         Fu[2, 0] = 0.0
70         Fu[2, 1] = dt
71
72     else:
73         Fx[0, 2] = -v * dt * math.sin(th)
74         Fx[1, 2] = v * dt * math.cos(th)
75
76         Fu[0, 0] = dt * math.cos(th)
77         Fu[0, 1] = -0.5 * v * dt ** 2 * math.sin(th)
78
79         Fu[1, 0] = dt * math.sin(th)

```

```

78         Fu[1, 1] = 0.5 * v * dt ** 2 * math.cos(th)
79
80         Fu[2, 0] = 0.0
81         Fu[2, 1] = dt
82
83     return Fx, Fu
84
85
86 def ekf_predict(x, P, u, Q, dt):
87     Fx, Fu = jacobians_motion(x, u, dt)
88
89     x_pred = motion_model(x, u, dt)
90     P_pred = Fx @ P @ Fx.T + Fu @ Q @ Fu.T
91     P_pred = 0.5 * (P_pred + P_pred.T)
92
93     return x_pred, P_pred
94
95
96 def measurement_model(x, beacon):
97     dx = beacon[0] - x[0]
98     dy = beacon[1] - x[1]
99
100     q = dx ** 2 + dy ** 2
101     q = max(q, 1e-12)
102
103     distance = math.sqrt(q)
104     bearing = wrap_angle(math.atan2(dy, dx) - x[2])
105
106     return np.array([distance, bearing], dtype=float)
107
108
109 def jacobian_measurement(x, beacon):
110     dx = beacon[0] - x[0]
111     dy = beacon[1] - x[1]
112
113     q = dx ** 2 + dy ** 2
114     q = max(q, 1e-12)
115
116     distance = math.sqrt(q)
117
118     H = np.zeros((2, 3))
119
120     H[0, 0] = -dx / distance
121     H[0, 1] = -dy / distance
122     H[0, 2] = 0.0
123
124     H[1, 0] = dy / q
125     H[1, 1] = -dx / q
126     H[1, 2] = -1.0
127
128     return H
129
130
131 def ekf_update(x, P, z, beacon, R):
132     h = measurement_model(x, beacon)
133     H = jacobian_measurement(x, beacon)
134
135     y = z - h
136     y[1] = wrap_angle(y[1])
137

```

```

138     S = H @ P @ H.T + R
139     S_inv = np.linalg.inv(S)
140
141     K = P @ H.T @ S_inv
142
143     x_new = x + K @ y
144     x_new[2] = wrap_angle(x_new[2])
145
146     I = np.eye(3)
147
148     P_new = (I - K @ H) @ P @ (I - K @ H).T + K @ R @ K.T
149     P_new = 0.5 * (P_new + P_new.T)
150
151     nis = float(y.T @ S_inv @ y)
152
153     return x_new, P_new, nis
154
155
156 def get_pose_xytheta(obj):
157     p = sim.getObjectPosition(obj, sim.handle_world)
158     e = sim.getObjectOrientation(obj, sim.handle_world)
159
160     return np.array([p[0], p[1], e[2]], dtype=float)
161
162
163 def get_beacon_positions():
164     b1 = sim.getObject('/Floor/b1')
165     b2 = sim.getObject('/Floor/b2')
166     b3 = sim.getObject('/Floor/b3')
167     b4 = sim.getObject('/Floor/b4')
168
169     beacons = []
170
171     for h in [b1, b2, b3, b4]:
172         p = sim.getObjectPosition(h, sim.handle_world)
173         beacons.append(np.array([p[0], p[1]], dtype=float))
174
175     return beacons
176
177
178 def command_profile(t):
179     v = 0.35
180     w = 0.45 * math.sin(0.15 * t)
181
182     if 35.0 < t < 50.0:
183         w = w + 0.35
184
185     if 75.0 < t < 90.0:
186         w = w - 0.40
187
188     return v, w
189
190
191 def get_wheel_speeds(dt):
192     global prev_left_pos, prev_right_pos
193
194     left_pos = sim.getJointPosition(leftMotor)
195     right_pos = sim.getJointPosition(rightMotor)
196
197     if prev_left_pos is None:

```

```

198     prev_left_pos = left_pos
199     prev_right_pos = right_pos
200     return 0.0, 0.0
201
202     d_left = wrap_angle(left_pos - prev_left_pos)
203     d_right = wrap_angle(right_pos - prev_right_pos)
204
205     wl = d_left / dt
206     wr = d_right / dt
207
208     prev_left_pos = left_pos
209     prev_right_pos = right_pos
210
211     return wl, wr
212
213
214 def state_error(x_est, x_true):
215     e = x_est - x_true
216     e[2] = wrap_angle(e[2])
217
218     return e
219
220
221 def sysCall_init():
222     global sim
223     global robotBase, leftMotor, rightMotor
224     global beacons
225     global x_ekf, P_ekf, x_odom
226     global t_prev, rows
227     global prev_left_pos, prev_right_pos
228
229     sim = require('sim')
230
231     random.seed(1)
232     np.random.seed(1)
233
234     robotBase = sim.getObject('/PioneerP3DX')
235     leftMotor = sim.getObject('/PioneerP3DX/leftMotor')
236     rightMotor = sim.getObject('/PioneerP3DX/rightMotor')
237
238     beacons = get_beacon_positions()
239
240     true_pose = get_pose_xytheta(robotBase)
241
242     x_ekf = true_pose + np.array([0.08, -0.05, math.radians(4.0)], dtype=float)
243     x_ekf[2] = wrap_angle(x_ekf[2])
244
245     x_odom = x_ekf.copy()
246     P_ekf = P0.copy()
247
248     t_prev = sim.getSimulationTime()
249
250     rows = []
251
252     prev_left_pos = None
253     prev_right_pos = None
254
255     print('LAB2 EKF started')
256     print('Robot: /PioneerP3DX')
257     print('Left motor: /PioneerP3DX/leftMotor')

```

```

258     print('Right motor: /Pioneer3DX/rightMotor')
259     print('Beacons: /Floor/b1, /Floor/b2, /Floor/b3, /Floor/b4')
260
261
262 def sysCall_actuation():
263     t = sim.getSimulationTime()
264
265     v_cmd, w_cmd = command_profile(t)
266
267     wl_cmd = (v_cmd - 0.5 * WHEEL_BASE * w_cmd) / WHEEL_RADIUS
268     wr_cmd = (v_cmd + 0.5 * WHEEL_BASE * w_cmd) / WHEEL_RADIUS
269
270     sim.setJointTargetVelocity(leftMotor, wl_cmd)
271     sim.setJointTargetVelocity(rightMotor, wr_cmd)
272
273     if t >= MAX_TIME:
274         sim.stopSimulation()
275
276
277 def sysCall_sensing():
278     global x_ekf, P_ekf, x_odom
279     global t_prev, rows
280
281     t = sim.getSimulationTime()
282     dt = t - t_prev
283
284     if dt <= 0.0:
285         return
286
287     t_prev = t
288
289     true_pose = get_pose_xytheta(robotBase)
290
291     wl, wr = get_wheel_speeds(dt)
292
293     wl_meas = wl + random.gauss(0.0, SIGMA_WHEEL)
294     wr_meas = wr + random.gauss(0.0, SIGMA_WHEEL)
295
296     v_meas = 0.5 * WHEEL_RADIUS * (wr_meas + wl_meas)
297     w_meas = (WHEEL_RADIUS / WHEEL_BASE) * (wr_meas - wl_meas)
298
299     u_meas = np.array([v_meas, w_meas], dtype=float)
300
301     x_odom = motion_model(x_odom, u_meas, dt)
302
303     x_ekf, P_ekf = ekf_predict(x_ekf, P_ekf, u_meas, Q_CONTROL, dt)
304
305     nis_values = []
306
307     for beacon in beacons:
308         z_true = measurement_model(true_pose, beacon)
309
310         z_range = z_true[0] + random.gauss(0.0, SIGMA_RANGE)
311         z_bearing = wrap_angle(z_true[1] + random.gauss(0.0, SIGMA_BEARING))
312
313         z = np.array([z_range, z_bearing], dtype=float)
314
315         x_ekf, P_ekf, nis = ekf_update(x_ekf, P_ekf, z, beacon, R_BEACON)
316
317         nis_values.append(nis)

```

```

318
319 e = state_error(x_ekf, true_pose)
320
321 try:
322     nees = float(e.T @ np.linalg.inv(P_ekf) @ e)
323 except Exception:
324     nees = float('nan')
325
326 nis_mean = float(np.mean(nis_values))
327
328 rows.append({
329     't': t,
330
331     'x_true': true_pose[0],
332     'y_true': true_pose[1],
333     'theta_true': true_pose[2],
334
335     'x_odom': x_odom[0],
336     'y_odom': x_odom[1],
337     'theta_odom': x_odom[2],
338
339     'x_ekf': x_ekf[0],
340     'y_ekf': x_ekf[1],
341     'theta_ekf': x_ekf[2],
342
343     'Pxx': P_ekf[0, 0],
344     'Pxy': P_ekf[0, 1],
345     'Pyx': P_ekf[1, 0],
346     'Pyy': P_ekf[1, 1],
347
348     'v_meas': v_meas,
349     'w_meas': w_meas,
350
351     'nis': nis_mean,
352     'nees': nees
353 })
354
355
356 def sysCall_cleanup():
357     if len(rows) == 0:
358         print('No data to save')
359         return
360
361     scene_path = sim.getStringParam(sim.stringparam_scene_path)
362
363     if scene_path is None or scene_path == '':
364         scene_path = os.getcwd()
365
366     out_path = os.path.join(scene_path, 'lab2_ekf_log.csv')
367
368     fieldnames = [
369         't',
370
371         'x_true',
372         'y_true',
373         'theta_true',
374
375         'x_odom',
376         'y_odom',
377         'theta_odom',

```

```

378         'x_ekf',
379         'y_ekf',
380         'theta_ekf',
381
382         'Pxx',
383         'Pxy',
384         'Pyx',
385         'Pyy',
386         'Ptt',
387
388         'v_meas',
389         'w_meas',
390
391         'nis',
392         'nees'
393     ]
394
395     with open(out_path, 'w', newline='') as f:
396         writer = csv.DictWriter(f, fieldnames=fieldnames)
397         writer.writeheader()
398         writer.writerows(rows)
399
400     print('CSV saved:', out_path)

```

Listing 2: Построение графиков

```

1  import argparse
2  import math
3  from pathlib import Path
4
5  import numpy as np
6  import pandas as pd
7  import matplotlib.pyplot as plt
8  from matplotlib.patches import Ellipse
9
10
11 def wrap_angle(a):
12     return (a + np.pi) % (2.0 * np.pi) - np.pi
13
14
15 def rmse(a):
16     a = np.asarray(a, dtype=float)
17     return float(np.sqrt(np.mean(a * a)))
18
19
20 def add_cov_ellipse(ax, x, y, P2, nsig=2.0):
21     vals, vecs = np.linalg.eigh(P2)
22     vals = np.maximum(vals, 1e-12)
23
24     order = vals.argsort()[::-1]
25     vals = vals[order]
26     vecs = vecs[:, order]
27
28     angle = math.degrees(math.atan2(vecs[1, 0], vecs[0, 0]))
29
30     width = 2.0 * nsig * np.sqrt(vals[0])
31     height = 2.0 * nsig * np.sqrt(vals[1])
32
33     ellipse = Ellipse(
34         xy=(x, y),

```

```

35         width=width,
36         height=height,
37         angle=angle,
38         fill=False,
39         linewidth=1.0
40     )
41
42     ax.add_patch(ellipse)
43
44
45 def process_file(csv_path, out_dir):
46     csv_path = Path(csv_path)
47     out_dir = Path(out_dir)
48     out_dir.mkdir(parents=True, exist_ok=True)
49
50     df = pd.read_csv(csv_path)
51
52     ex_ekf = df["x_ekf"] - df["x_true"]
53     ey_ekf = df["y_ekf"] - df["y_true"]
54     eth_ekf = wrap_angle(df["theta_ekf"].to_numpy() - df["theta_true"].to_numpy())
55
56     ex_odom = df["x_odom"] - df["x_true"]
57     ey_odom = df["y_odom"] - df["y_true"]
58     eth_odom = wrap_angle(df["theta_odom"].to_numpy() - df["theta_true"].to_numpy())
59
60     pos_err_ekf = np.sqrt(ex_ekf ** 2 + ey_ekf ** 2)
61     pos_err_odom = np.sqrt(ex_odom ** 2 + ey_odom ** 2)
62
63     summary = {
64         "file": csv_path.name,
65         "rmse_pos_ekf_m": rmse(pos_err_ekf),
66         "rmse_pos_odom_m": rmse(pos_err_odom),
67         "rmse_theta_ekf_rad": rmse(eth_ekf),
68         "rmse_theta_odom_rad": rmse(eth_odom),
69         "mean_nis": float(np.nanmean(df["nis"])),
70         "mean_nees": float(np.nanmean(df["nees"])),
71     }
72
73     stem = csv_path.stem
74
75     fig, ax = plt.subplots(figsize=(8, 6))
76
77     ax.plot(df["x_true"], df["y_true"], label="истинная траектория")
78     ax.plot(df["x_odom"], df["y_odom"], label="одометрия")
79     ax.plot(df["x_ekf"], df["y_ekf"], label="EKF")
80
81     step = max(len(df) // 20, 1)
82
83     for i in range(0, len(df), step):
84         P2 = np.array([
85             df["Pxx"].iloc[i], df["Pxy"].iloc[i],
86             df["Pxy"].iloc[i], df["Pyx"].iloc[i],
87         ])
88
89         add_cov_ellipse(
90             ax,
91             df["x_ekf"].iloc[i],
92             df["y_ekf"].iloc[i],

```



```

93         P2,
94         nsig=2.0
95     )
96
97     ax.set_xlabel("x, м")
98     ax.set_ylabel("y, м")
99     ax.set_title("Траектории и ковариационные эллипсы EKF")
100    ax.axis("equal")
101    ax.grid(True)
102    ax.legend()
103
104    fig.tight_layout()
105    fig.savefig(out_dir / f"{stem}_trajectory.png", dpi=200)
106    plt.close(fig)
107
108    fig, ax = plt.subplots(figsize=(8, 5))
109
110    ax.plot(df["t"], pos_err_odom, label="одометрия")
111    ax.plot(df["t"], pos_err_ekf, label="EKF")
112
113    ax.set_xlabel("t, с")
114    ax.set_ylabel("ошибка положения, м")
115    ax.set_title("Ошибка положения")
116    ax.grid(True)
117    ax.legend()
118
119    fig.tight_layout()
120    fig.savefig(out_dir / f"{stem}_position_error.png", dpi=200)
121    plt.close(fig)
122
123    fig, ax = plt.subplots(figsize=(8, 5))
124
125    ax.plot(df["t"], np.sqrt(df["Pxx"]), label="sigma x")
126    ax.plot(df["t"], np.sqrt(df["Pyy"]), label="sigma y")
127    ax.plot(df["t"], np.sqrt(df["Ptt"]), label="sigma theta")
128
129    ax.set_xlabel("t, с")
130    ax.set_ylabel("стандартное отклонение")
131    ax.set_title("Ковариация EKF")
132    ax.grid(True)
133    ax.legend()
134
135    fig.tight_layout()
136    fig.savefig(out_dir / f"{stem}_covariance.png", dpi=200)
137    plt.close(fig)
138
139    fig, ax = plt.subplots(figsize=(8, 5))
140
141    ax.plot(df["t"], df["nis"], label="NIS")
142    ax.plot(df["t"], df["nees"], label="NEES")
143
144    ax.set_xlabel("t, с")
145    ax.set_ylabel("значение")
146    ax.set_title("NIS и NEES")
147    ax.grid(True)
148    ax.legend()
149
150    fig.tight_layout()
151    fig.savefig(out_dir / f"{stem}_nis_nees.png", dpi=200)
152    plt.close(fig)

```

```

153
154     return summary
155
156
157 def main():
158     parser = argparse.ArgumentParser()
159     parser.add_argument("csv", nargs="+")
160     parser.add_argument("--out", default="plots_lab2")
161
162     args = parser.parse_args()
163
164     summaries = []
165
166     for csv_file in args.csv:
167         summaries.append(process_file(csv_file, args.out))
168
169     summary_df = pd.DataFrame(summaries)
170
171     print(summary_df.to_string(index=False))
172
173     Path(args.out).mkdir(parents=True, exist_ok=True)
174     summary_df.to_csv(Path(args.out) / "summary_metrics.csv", index=False)
175
176
177 if __name__ == "__main__":
178     main()

```